

ЛЕКЦИЯ 05

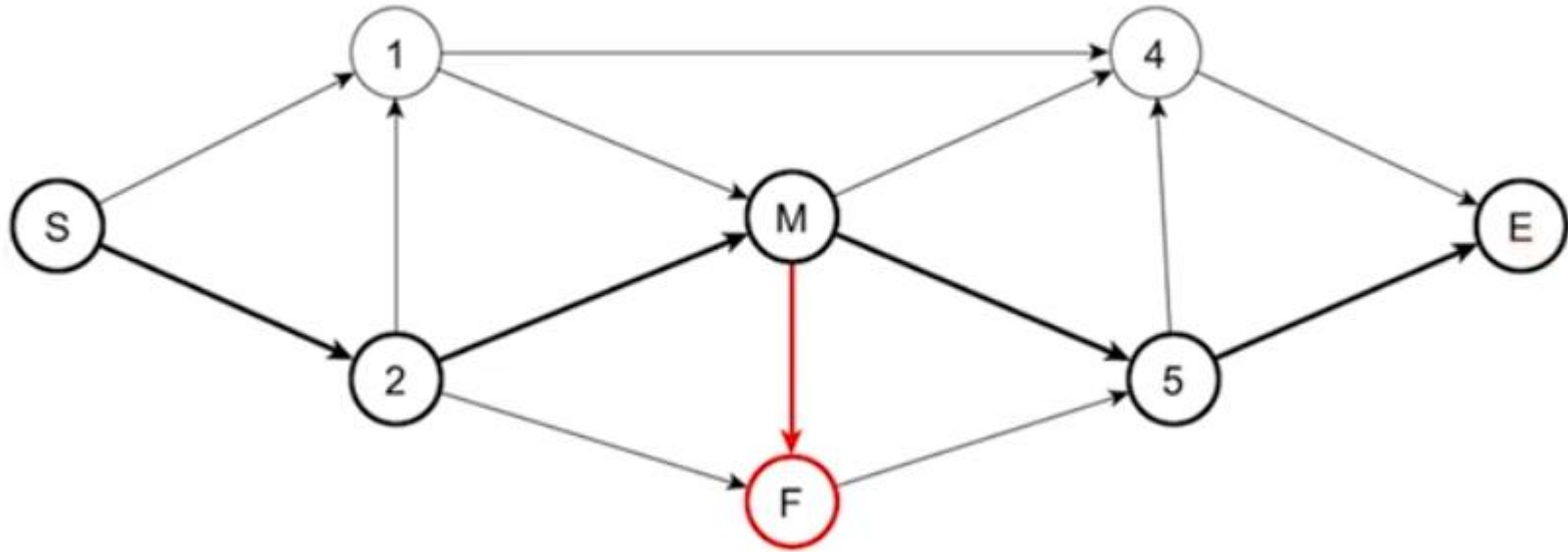
ПРИНЦИП ОПТИМАЛЬНОСТИ БЕЛМАНА

СИСТЕМЫ ИСКУССТВЕННОГО
ИНТЕЛЛЕКТА

ЛЕКТОР ФУРМАВНИН С.А.



ПРИНЦИП ОПТИМАЛЬНОСТИ БЕЛЛМАНА



- Если траектория от S до E оптимальна, то и траектория от S до M тоже.
- Но не наоборот. Траектории от S до M и от M до F вместе не дают оптимальную траекторию

ФУНКЦИОНАЛЬНОЕ УРАВНЕНИЕ БЕЛМАНА

- «*An optimal policy has the property that whatever the initial state and initial decisions are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision*» [Bellman]
- Изменение состояния: $x_t \rightarrow x_{t+1} = T(x_t, a_t)$ with pay – off $F(x_t, a_t)$
- Обозначим $V(x_0) = \max_{a_0 \dots a_T} \sum_t F(x_t, a_t)$
- Тогда $V(x_t) = \max_{a_t} \{F(x_t, a_t) + V(T(x_t, a_t))\}$
- Простыми словами: любой кусок оптимальной траектории является оптимальной траекторией

ЗАДАЧА РАЗМЕНА МОНЕТ

- Пусть есть монеты номиналами $c_1 \dots c_k$ и необходимо разменять сумму N , взяв наименьшее количество монет
- x_t - это оставшаяся сумма для размена
- $a_t \in \{c_1 \dots c_k\}$ - это решение сколько мы берём на каждом шаге

$$x_{t+1} = x_t - a_t$$

$$F(x_t, a_t) = 1$$

- мы должны минимизировать количество шагов, т.е. задача на минимизацию, а не на максимизацию
- Как записать функцию Беллмана?

ЗАДАЧА РАЗМЕНА МОНЕТ

- Пусть есть монеты номиналами $c_1 \dots c_k$ и необходимо разменять сумму N , взяв наименьшее количество монет
- Запишем функциональное уравнение Беллмана
- $V(t) = \min(V(t - c_1), \dots, V(t - c_k)) + 1$
- Иными словами: на каждом шаге мы берем такую монету, чтобы функция Беллмана была минимальной на прошлом шаге.
- Нас не волнует то, что тут $\min$, так как $\max(f(t)) = -\min(-f(t))$ и наоборот

ЗАДАЧА РАЗМЕНА МОНЕТ

- Приложение к размену монет: пусть есть монеты номиналами c_1, c_2, c_3 и необходимо разменять сумму N
- Заведем таблицу $V(x_k)$ от 0 до N и протабулируем оптимальные размены. Пусть номиналы 1, 3, 4 и надо разменять 1, используя минимальное количество монет

k	1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	---	----

V	1	2								
---	---	---	--	--	--	--	--	--	--	--

ЗАДАЧА РАЗМЕНА МОНЕТ

- Приложение к размену монет: пусть есть монеты номиналами c_1, c_2, c_3 и необходимо разменять сумму N
- Заведем таблицу $V(x_k)$ от 0 до N и протабулируем оптимальные размены. Пусть номиналы 1, 3, 4 и надо разменять 1, используя минимальное количество монет

k	1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	---	----

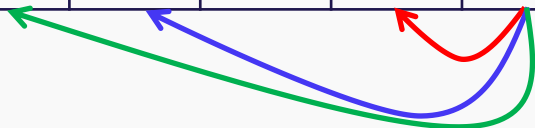
V	1	2	1	1						
---	---	---	---	---	--	--	--	--	--	--

ЗАДАЧА РАЗМЕНА МОНЕТ

- Приложение к размену монет: пусть есть монеты номиналами c_1, c_2, c_3 и необходимо разменять сумму N
- Заведем таблицу $V(x_k)$ от 0 до N и протабулируем оптимальные размены. Пусть номиналы 1, 3, 4 и надо разменять 1, используя минимальное количество монет

k	1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	---	----

V	1	2	1	1	2					
---	---	---	---	---	---	--	--	--	--	--



ЗАДАЧА РАЗМЕНА МОНЕТ

- Приложение к размену монет: пусть есть монеты номиналами c_1, c_2, c_3 и необходимо разменять сумму N
- Заведем таблицу $V(x_k)$ от 0 до N и протабулируем оптимальные размены. Пусть номиналы 1, 3, 4 и надо разменять 1, используя минимальное количество монет

k	1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	---	----

V	1	2	1	1	2	2	2	2	3	3
---	---	---	---	---	---	---	---	---	---	---



ВОСХОДЯЩЕЕ И НИСХОДЯЩЕЕ РЕШЕНИЕ

```
int calc_change(int n, int m, int *changes, int *V) {  
    int i;  
    if (n == 0) return 0;  
    if (n < 0) return INT_MAX;  
    if (V[n] != INT_MAX) return V[n];  
    for (i = 0; i < m; ++i) {  
        int CT = calc_change(n - changes[i], m, changes, V);  
        if (CT == INT_MAX) continue;  
        V[n] = min(V[n], CT + 1);  
    }  
    return V[n];  
}
```

$$V(t) = \min(V(t - c_1), \dots, V(t - c_k)) + 1$$

ЗАДАЧА О РЮКЗАКЕ

Вам необходимо написать программу, которая, считывая со стандартного ввода:

- количество вещей
- общий вес, входящий в рюкзак
- Вес каждой вещи w_i
- Стоимость каждой вещи v_i

выдавало бы наибольшую стоимость вещей, которые можно положить в рюкзак.

Например, для 4 6 4 3 2 1 5 4 3 2 ответом будет $2 + 3 + 4 = 9$

ФУНКЦИОНАЛЬНОЕ УРАВНЕНИЕ БЕЛМАНА

$$V(0, w) = 0$$

$$V(i, w) = V(i - 1, w) \text{ для } w < w_i$$

$$V(i, w) = \max(V(i - 1, w), V(i - 1, w - w_i) + v_i) \text{ для } w > w_i$$

i	v	w
1	5	4
2	4	3
3	3	2
4	2	1

	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	0	0	0	5	5	5
2	0	0	0	4	5	5	5
3	0	0	3	4	5	7	8
4	0	2	3	5	6	7	9

ОБСУЖДЕНИЕ

- Довольно сложно было догадаться в задаче о рюкзаке, что функция Беллмана будет двумерной
- У нас там две степени свободы: каждую вещь можно взять или не взять (первый уровень) с учетом того, как взяты остальные вещи (второй уровень)

РАССТОЯНИЕ РЕДАКТИРОВАНИЯ

Как перейти от слова **spoon** к слову **sponge**?

Что можно делать:

- Вставлять букву в любое место ($\text{cost} == 1$)
- Удалять любую букву ($\text{cost} == 1$)
- Изменять любую букву ($\text{cost} == 2$)

Нужно найти последовательность действий с минимальной ценой

spoon → **sponn** (+2) → **spong** (+2) → **sponge** (+1), $\text{cost} == 5$

spoon → **spon** (+1) → **spong** (+1) → **sponge** (+1), $\text{cost} == 3$

Можем ли мы применить динамическое программирование?

ФУНКЦИОНАЛЬНОЕ УРАВНЕНИЕ БЕЛМАНА

Любая часть оптимальной траектории является оптимальной траекторией

$$V(i, j) = \min \begin{cases} V(i-1, j) + 1 \\ V(i, j-1) + 1 \\ V(i-1, j-1) + K(i, j) \end{cases}$$

$$K(i, j) = \begin{cases} 0, S_1(i) = S_2(j) \\ 2, S_1(i) \neq S_2(j) \end{cases}$$

Теперь задача сводится к тому, чтобы найти $V(5, 6)$

	#	S	P	O	N	G	E
#	0	1	2	3	4	5	6
S	1	$V(1,1)$					
P	2						
O	3						
O	4						
N	5						$V(5,6)$

Потренируемся заполнять таблицу?

ФУНКЦИОНАЛЬНОЕ УРАВНЕНИЕ БЕЛМАНА

Любая часть оптимальной траектории является оптимальной траекторией

$$V(i, j) = \min \begin{cases} V(i-1, j) + 1 \\ V(i, j-1) + 1 \\ V(i-1, j-1) + K(i, j) \end{cases}$$

$$K(i, j) = \begin{cases} 0, S_1(i) = S_2(j) \\ 2, S_1(i) \neq S_2(j) \end{cases}$$

Теперь задача сводится к тому, чтобы найти $V(5, 6)$

Нашли!

	#	S	P	O	N	G	E
#	0	1	2	3	4	5	6
S	1	0	1	2	3	4	5
P	2	1	0	1	2	3	4
O	3	2	1	0	1	2	3
O	4	3	2	1	2	3	4
N	5	4	3	2	1	2	3

ФУНКЦИОНАЛЬНОЕ УРАВНЕНИЕ БЕЛМАНА

- Можем ли мы в задаче о расстоянии редактирования восстановить решение?
- Обратим внимание: может быть больше одного решения весом 3
- Также следует заметить, что у нас есть некоторый выбор: оптимизировать память или восстанавливать решение

	#	S	P	O	N	G	E
#	0	1	2	3	4	5	6
S	1	0	1	2	3	4	5
P	2	1	0	1	2	3	4
O	3	2	1	0	1	2	3
O	4	3	2	1	2	3	4
N	5	4	3	2	1	2	3

ВЕРНЕМСЯ К ПРЕДЕЛАМ РЕГУЛЯРНОСТИ

Мы хотели бы сматчить все правильные скобочные выражения и только их.

Для этого невозможно написать регулярное выражение, но можно написать грамматику

ГРАММАТИКА

По определению грамматика состоит из продукций $\alpha \rightarrow \beta$
 $A \rightarrow A + A | (A) | A.A | A^* | a | b | c | \varepsilon$

Нетерминальные символы в общем случае могут стоять и слева и справа, терминальные — только справа

Для языка регулярных выражений на $\{a, b, c\}$

- Терминалы: $a, b, c, +, (,), *$
- Нетерминалы: пока только A

Обратим внимание: во всех продукциях языка регулярных выражений у нас слева всего один нетерминал

ПРИМЕР ПРОСТОЙ ГРАММАТИКИ

Контекстно-свободной грамматикой называется такая, которую можно представить так, чтобы слева в каждой продукции был ровно один нетерминал

Любой регулярный язык тривиально является контекстно-свободным

Язык $L_{parm} = \{a^m b^n c a^m b^n\}$ не является контекстно-свободным

Грамматика для скобочных выражений.

$$B \rightarrow (B) B$$

Возможные порождаемые скобки – любые скобочные выражения

$(((()))) , (((())) ())$

ПОРОЖДЕНИЕ

Порождаем все скобочные выражения

$$V \rightarrow (B) V$$

Сделаем левое порождение $((())())$

$$V \rightarrow (B)V \rightarrow ((B)V)V \rightarrow ((\)B)V \rightarrow ((\)(B)V)V \rightarrow ((\)((B)V)V)V$$

Чтобы не порождать, а распознавать выражения, приведем грамматику в нормальную форму

НОРМАЛЬНАЯ ФОРМА ХОМСКОГО

Разрешены только следующего вида продукции

$$A \rightarrow a, A \rightarrow BC, S \rightarrow \varepsilon$$

Основная идея в замене $A \rightarrow aB$ на $A \rightarrow XB, X \rightarrow a$

Пример: $B \rightarrow (B)B|\varepsilon$

Как мы могли бы распознать строку $((() (())))$?

$S \rightarrow LZ_1|LR|\varepsilon$
 $B \rightarrow LZ_1|LR$
 $Z_1 \rightarrow BZ_2|BR$
 $Z_2 \rightarrow RB$
 $R \rightarrow)$
 $L \rightarrow ($

Благодаря свойствам нормальной формы, можно привести её в нетерминалы

L	L	R	L	L	R	R	R
(	(	)	(	(	)	)	)

ИДЕЯ ДЛЯ АЛГОРИТМА ПРИНАДЛЕЖНОСТИ

Далее попробуем комбинировать подстроки длины 2, длины 3 и так далее

Пусть мы ищем возможный вывод $S \rightarrow LLRLLRRR$

В этом случае мы должны рассматривать $X_i \rightarrow X_j X_k$

У нас три параметра: начало диапазона, длина и продукция

$S \rightarrow LZ_1 | LR | \varepsilon$

$B \rightarrow LZ_1 | LR$

$Z_1 \rightarrow BZ_2 | BR$

$Z_2 \rightarrow RB$

$R \rightarrow)$

$L \rightarrow ($

	B			B			
L	L	R	L	L	R	R	R
(	(	)	(	(	)	)	)

ПРИНЦИП ОПТИМАЛЬНОСТИ

Для алгоритма СΥΚ принцип оптимальности примет следующий вид

$$V(1, i, j) = true$$
$$V(k, i, a) = \max(V(p, i, b) \wedge V(k - p, i + p, c)), \text{ где } X_a \rightarrow X_b X_c$$

Это сводится (снова) к заполнению двумерной таблицы. Но на этот раз мы должны рассматривать также диапазон индексов p от единицы до длины строки.

Что сразу приводит нас к простому псевдокоду

АЛГОРИТМ КОКА-ЯНГЕРА-КАСАМИ

Для строки с n символами и для k продукций, после начальной инициализации, алгоритм следующий:

```
for (l = 2, l < n; ++l)
    for (s = 1; s < n - l + 1; ++s)
        for (p = 1; p < l - 1; ++p)
            for each production  $Xa \rightarrow Xb Xc$ 
                 $V[l, s, a] = \max(V[l, s, a],$ 
                                    $V[p, s, b], \&\& V[l - p, s + p, c])$ 
```

Реализуйте этот код на языке C++

Если в итоге $V[n, 1, 1] = \text{true}$, то слово принадлежит языку

ФОРМИРОВАНИЕ ИНТУИЦИИ

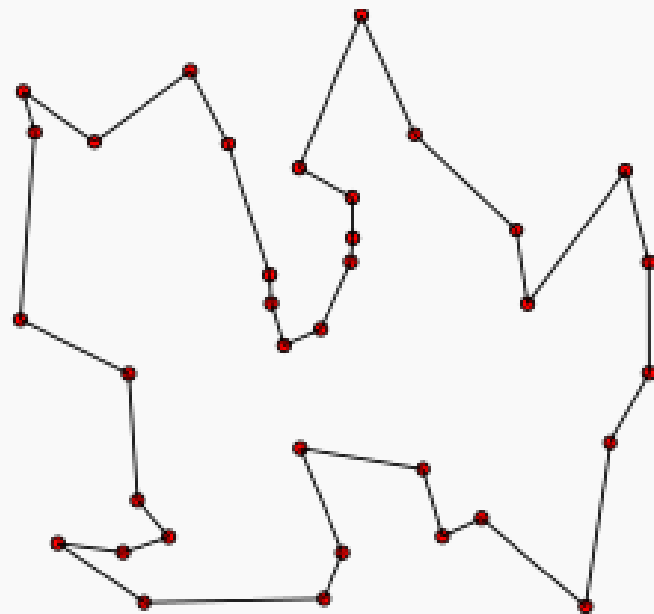
Не все задачи можно эффективно решить динамическим программированием

Пример: travelling salesman problem

Можно ли написать функциональное уравнение Беллмана? В общем можно

$$V(S, k) = \min(V(S - \{k\}, m) + d(m, k))$$

Увы, решение при этом будет $O(n^2 2^n)$ по времени и по памяти



РЕКОМЕНДОВАННАЯ ЛИТЕРАТУРА

1. Попов Э.В. и др. Искусственный интеллект. Т.1. – М.: Радио и связь, 1990.
2. Нильсон Н. Искусственный интеллект : Пер. с англ. - М.: Мир, 1973.
3. Адаменко А.Н., Кучуков А.М. Логическое программирование и Visual Prolog. – СПб.: БХВПетербург, 2003.
4. Хювенен Э., Сеппянен Й. Мир Лиспа. Т.1. Пер. с финск. – М.: Мир, 1990.